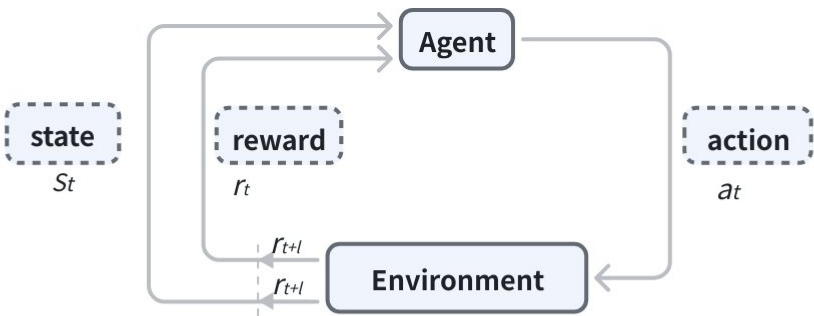


强化学习



PPO近端策略优化、DPO直接偏好优化、DAPO直接对齐偏好优化、GRPO组相对策略优化、RLEF

方法名	是否需要人工偏好	是否训练奖励模型	是否策略训练	优点	缺点
PPO 近端策略优化	✅ 是	✅ 是	✅ 是	高质量策略微调	成本高、流程复杂
DPO 直接偏好优化	✅ 是（成对偏好）	❌ 否	❌ 否	训练效率高	不够灵活、稳定性差
GRPO 组相对策略优化	✅ 是（成对偏好）	❌ 否	❌ 否	收敛更稳定	仍依赖偏好数据
DAPO	❌ 否（自动构造）	❌ 否	❌ 否	无需人工偏好	自评分可能误导模型
RLEF	❌ 否（专家打分）	❌ 否	✅ 是	无需标注数据	奖励设计门槛高

方法	全称	核心思想
PPO	Proximal Policy Optimization	基于策略梯度 + 奖励模型，优化输出策略以最大化奖励
DPO	Direct Preference Optimization	用人类偏好对比训练策略，最大化偏好回答的概率
GRPO	Generalized Rejection Preference Optimization	DPO 的泛化版本，用更稳定的损失优化偏好
DAPO	Direct Alignment Preference Optimization	类似 DPO，但偏好由语言模型自身评估生成，省去人工
RLEF	Reinforcement Learning with Expert Feedback	用专家规则、评估器等替代人类打分做奖励函数，用 RL 优

维度 🧩	PPO	DPO	GRPO	DAPO	RLEF
✅ 是否训练策略模型	是	是	是	是	是
🧠 优化方式	策略梯度 (RL)	偏好对比监督学习	多样性排序对比学习 (Groupwise DPO)	自动构造偏好对比训练	偏好+模仿混合策略训练
🏆 奖励来源	人类偏好训练的奖励模型 (RM)	成对偏好数据 (人工或自动)	同 DPO (排序级)	策略模型对回答的 log-prob 差值	偏好样本 + 专家示范 (或规则)
📦 是否需要 Reward Model	✅ 是	❌ 否	❌ 否	❌ 否	❌ 否 (选配规则/得分器)
👤 是否需要人工偏好数据	✅ 是 (训练 RM)	✅ 是 (训练对比)	✅ 是 (训练排序)	❌ 否 (用模型自动对比)	✅ 推荐 (也可规则生成)
🎯 训练目标	最大化奖励期望	最大化人类偏好回答概率	更稳定地优化偏好排序分布	模拟人类偏好差异	同时对齐人类偏好与专家行为
📈 收敛速度	慢	快	快且更稳	快	中等 (依任务)
💰 训练成本	高 (RM + RL)	中 (监督训练)	中 (排序扩展)	低 (自动打分 + 训练)	中 (偏好 + imitation)
⚠️ 缺点	成本高、训练不稳定	偏好数据质量敏感	超参调节复杂、样本利用率有限	自动偏好准确性有限	多损失组合复杂、调参难
✅ 优点	奖励灵活，泛化强	简洁高效，无需 RM	更稳定，泛化能力更强	自动生成偏好，无需人工标注	同时学习偏好与专家行为，效果鲁棒
🚀 应用代表	ChatGPT (RLHF 阶段)	Anthropic Claude (DPO)	Anthropic 实验性排序优化模型	DAPO 官方实验模型	特定场景下的自研系统 (如医疗/法律)

 **强化学习**目标是让智能体 (agent) 通过与环境 (environment) 的交互来学习最优的行为策略 (policy)，从而最大化某个累积回报 (cumulative reward)。核心思想是通过试错和反馈的机制，找到在每个情境下的最优决策。

强化学习的**优化目标**是通过选择策略来最大化累积奖励。具体来说，智能体的目标是寻找一个最优策略，使得它在各个状态下的累积回报最大。形式上，可以用**价值函数** (Value Function) 或**动作价值函数** (Q函数) 来表示。

🌟 **策略模型**：其实就是语言模型本身，我们让它通过学习“偏好”来调整自己的输出行为。

在强化学习中：

- **策略 (Policy)** 通常表示为：
 $\pi(a | s)$
即：在状态 s 下采取动作 a 的概率。
- **策略模型** 就是这个 $\pi(a | s)$ 的建模器，通常是一个神经网络，用于生成策略。
- 在 RLHF / PPO / DPO 等方法中：
 - **状态 s** 是用户输入的 prompt (问题、对话、命令)
 - **动作 a** 是语言模型生成的回答 (也可以看作是一步一步生成的 token)
 - 所以语言模型就是一个“从输入 prompt 到输出回答的策略分布”

奖励

单步奖励

根据当前状态、动作和下一个状态由奖励模型得到的即时奖励，评估当前动作的好坏。

累计奖励

一条完整轨迹的单步奖励累积之和。

折扣奖励

平衡即时奖励和长期奖励之间的关系，使得智能体在决策时不仅要考虑当前的奖励，还要考虑未来的潜在奖励。

🌟 轨迹 Rollout

轨迹由一系列的状态、动作组成，代表一次完整的采样，即大模型生成一条完整的句子。

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1})$$

策略模型 vs 奖励模型

策略模型生成回答 → 奖励模型打分 → 强化学习优化策略模型

模型类型	定义	输入	输出	用途
 策略模型	表示“如何回答”的分布	prompt	回答	生成回答（训练目标）
 奖励模型	判断回答是否好	prompt + 回答	分数（reward）	指导策略模型的训练

一、PPO近端策略优化

2017 OpenAI

核心目标是避免策略更新中的剧烈变化

4+2理解法

四个模型：策略模型、价值模型、奖励模型、参考模型

- 策略模型：待优化的模型，参与参数更新
- 价值模型：计算当前动作和状态的期望回报，可由奖励模型或策略模型初始化而来，参与参数更新
- 奖励模型：计算当前动作的即时奖励，不参与参数更新
- 参考模型：由策略模型进行初始化，不参与参数更新，用于限制策略模型在优化时不偏离原始模型太远

两个损失：策略损失、价值损失

- 策略损失：用于优化策略模型
- 价值损失：用于优化价值模型

动机：Actor-Critic 方法有一定局限性

时间差分TD-Error优势估计不准：

$\delta_t = r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ r_t 是在t时刻获得的奖励， γ 是折扣因子(0~1], $V_{\pi}(s_t)$ 是策略的价值函数的估计

TD-Error依赖于当前策略的价值函数 V_{π} ，这个估计本身可能存在**偏差**，特别是在训练的早期阶段。

- **偏差 (Bias)**：反映策略更新的方向是否准确，这里的偏差主要来自于 V_π 。
- **方差 (Variance)**：反映策略更新的稳定性，这里的方差主要来自于采样噪音

👍 PPO近端策略优化

广义优势估计GAE：

用于解决强化学习中状态价值估计与实际回报计算之间的偏差问题，最终目的是更准确地估计智能体在每个状态下的“优势”，从而提升策略更新的稳定性和效率

$$A_\pi(s_t, a_t) = Q_\pi(s_t, a_t) - V_\pi(s_t)$$

$Q_\pi(s_t, a_t)$ 是动作价值函数表示在状态 s_t 下执行动作 a_t 的总收益期望，累积折扣奖励（回报）

$V_\pi(s_t)$ 是状态价值函数，表示在状态 s_t 下采取最优策略的平均收益期望。

- 如果奖励比期望的高， $A > 0$ 策略会增强这个动作，否则策略会减弱它。

GAE是通过在时间步上对 TD Error 进行加权累加

$$\begin{aligned} A_t &= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \\ &= \sum_{l=0}^{\infty} (\gamma\lambda)^l (r_{t+l} + \gamma V_\pi(s_{t+l+1}) - V_\pi(s_{t+l})) \end{aligned}$$

$\lambda \in [0, 1]$ 是GAE的衰减系数没控制偏差和方差之间的平衡

- λ 趋于0时， $A_t = r_t + \gamma V_\pi(s_{t+1}) - V_\pi(s_t)$ ，高偏差（其实就是时间差分TD-Error优势估计）
- λ 趋于1时，GAE退化为完整的蒙特卡洛估计，具有较低偏差但较高方差。

$$\begin{aligned} A_t &= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \\ &= \sum_{l=0}^{\infty} (\gamma\lambda)^l (r_{t+l} + \gamma V_\pi(s_{t+l+1}) - V_\pi(s_{t+l})) \\ &= r_t + \gamma V_\pi(s_{t+1}) - V_\pi(s_t) + \gamma r_{t+1} + \gamma^2 V_\pi(s_{t+2}) - \gamma V_\pi(s_{t+1}) + \dots \\ &= -V_\pi(s_t) + r_t + \lambda r_{t+1} + \dots \\ &= G_t - V_\pi(s_t) \end{aligned}$$

$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{T-t-1} r_{T-1}$ 是蒙特卡洛回报（从 t 时刻到终止状态的累积折扣奖励）

每个奖励 r_t 都可能包含噪声（如人类反馈的随机性、环境动态的不确定性）。这些噪声会随着时间步累积到 G_t 中，导致 G_t 的取值波动很大，从而导致 A_t 方差高

$$A_t = \delta_t + \gamma\lambda A_{t+1}$$

所以调节 λ 的大小，控制偏差-方差权衡，使策略梯度估计更稳定。

♥ PPO目标函数

1.显式约束：在目标函数中直接加入KL散度的惩罚项，将约束条件转化为惩罚项。

目标函数为：

$$L^{KL PEN}(\theta) = \mathbb{E}_t \left[r_t(\theta) A_t^{GAE(\gamma, \lambda)} - \beta D_{KL}(\pi_{\theta_{old}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t)) \right]$$

其中：

$\mathbb{E}_t$ 表示 “对时间步 t 的样本进行经验平均”

$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 是新旧策略的概率比；

$A_t^{GAE(\gamma, \lambda)}$ 是优势估计GAE；

β 是惩罚系数，控制KL散度对目标函数的影响。

显式约束是通过在优化过程中监控 KL 散度，根据实际值调整 β 的大小，实现对策略更新幅度的动态控制：

- 首先，对 D_{KL} 设置上下限，分别记为 $D_{\max}$ 和 $D_{\min}$ 。
- 当 $D_{KL} \geq D_{\max}$ 时，说明当前策略已经**偏离旧策略较远了**，这时应该增大 β ，把分布拉回来。
- 当 $D_{KL} \leq D_{\min}$ 时，说明当前策略很可能**找到了一条捷径**，即它只优化 KL 散度一项，而不去优化前面优势相关的项，所以这时应该减小 β ，降低惩罚项的影响。



- D_{KL} 是KL散度，用来衡量两个概率分布之间的“距离”或“差异”

$$D_{KL}(P||Q) = \sum_x P(x) \cdot \log \frac{P(x)}{Q(x)}$$

- $P(x)$ 是目标分布(如当前策略模型 π_{θ})
- $Q(x)$ 是参考分布(如旧策略模型 $\pi_{\theta_{old}}$)
- x 是有可能输出(如 $token$ 序列)

2.隐式约束：如果觉得KL散度的动态调整也很麻烦，还有一个更简单的 PPO-Clip 算法。它在目标函数中直接对概率比率 $r_t(\theta)$ 进行裁剪，避免了引入二阶导数或 KL 散度计算。

目标函数为：

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

其中：

$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ 表示将 $r_t(\theta)$ 限制在 $[1 - \epsilon, 1 + \epsilon]$ 内；

ϵ 是超参数，控制测控也变化范围，一般取值0.1或0.2；

$\min(*)$ 操作实现在优势函数为正的时候，仅对 $1 + \epsilon$ 裁剪；为负时，仅对 $1 - \epsilon$ 裁剪。

裁剪的目标就是防止 π_θ 偏离 $\pi_{\theta_{old}}$ ，偏离的时候就需要缩小更新的幅度了：

当优势 $A_t > 0$ 时，此时倾向于让 $\pi_\theta(a_t|s_t)$ 变大，因此 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 会变大：

- 如果两个分布相同，则 $r_t(\theta) = 1$ ，此时训练稳定，**无需裁剪**；
- 如果 $\pi_\theta(a_t|s_t)$ 在之前的更新过程中已经变大，则 $r_t(\theta) > 1$ ，**需要进行裁剪**，将其限制在 $1 + \epsilon$ 内防止越变越大，分布越偏越远；
- 如果 $\pi_\theta(a_t|s_t)$ 在之前的更新过程中变小了，则 $r_t(\theta) < 1$ ，但此时的更新会导致 $r_t(\theta) - > 1$ ，这是希望看到的变化，于是**不再对其进行裁剪**。

反之，当优势 $A_t < 0$ 时，也有类似的操作。

PPO算法实现

两个损失函数：

1.策略损失：策略网络更新目标就是最大化PPO的目标函数，策略网络的参数 θ 应该通过梯度上升法进行更新。

以clip为例， $L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$

步骤如下：

1. **计算概率比率：**对于每个经验池中的每个样本 (s_t, a_t) ，用当前策略进行推断后，计算 $r_t(\theta)$ 。
2. **计算裁剪后的目标：**根据经验中 A_t 的符号，对 $r_t(\theta)$ 进行裁剪，得到裁剪后的目标值。
3. **计算目标函数：**将裁剪后的目标值与未裁剪的目标值进行比较，取最小值作为最终的目标函数值。

2.价值损失：使用均方误差MSE作为损失函数，最小化如下目标函数：

$$L^{VF}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - V_t^{target})^2 \right]$$

具体计算步骤如下：

1. **计算 GAE 优势：**从轨迹的末端开始，对于每个样本 (s_t, a_t, r_t, s_{t+1}) 反向计算每个时刻的 A_t^{GAE} 。
2. **构造目标值：** $V_{target}(s_t) = V_{\phi_{old}}(s_t) + A_t^{GAE}$ ，算好后与经验存在一起备用。 ϕ_{old} 是旧的价值网络。
3. **计算均方误差：**计算价值网络输出 $V_\phi(s_t)$ 与目标值 V_t^{target} 之间的均方误差。

算法步骤：

1. 初始化策略网络 π_{θ} 和价值网络 V_{ϕ} ，设定超参数 ϵ 、 γ 、 λ 等。
2. 采集交互数据：使用旧策略网络 $\pi_{\theta_{old}}$ 与环境进行交互，收集一批轨迹数据 (s_t, a_t, r_t, s_{t+1}) 。
3. 计算优势：使用 GAE 计算每个时间步的优势估计 A_t 。

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

其中， $\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$ 使用旧价值网络计算。

4. 计算策略目标，更新策略网络：对于每个样本，计算概率比率 $r_t(\theta)$ 和目标函数 $L^{CLIP}(\theta)$ ，使用优化器更新策略网络参数 θ 。

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

5. 计算状态价值的目标：对于每个样本，使用旧价值网络计算：

$$V_{target}(s_t) = V_{\phi_{old}}(s_t) + A_t^{GAE} \quad \text{target 其实就是 GAE 计算的回报}$$

6. 更新价值网络：最小化均方误差损失，使用优化器更新价值网络参数。

$$L^{VF}(\phi) = \mathbb{E}_t [(V_{\phi}(s_t) - V_t^{target})^2]$$

7. 重复迭代：直到策略收敛或达到预定的停止条件。

🌟 RLHF-PPO: PPO 在 RLHF 中的应用

RLHF 通过训练一个**奖励模型**（Reward Model）来量化人类的偏好，并使用强化学习算法（如 PPO）来优化策略模型（Policy Model）

在 RLHF 的训练过程中，涉及到四个模型的协同工作：策略模型、参考模型、价值模型、奖励模型。这四个模型各自承担不同的角色，一起实现策略的优化：

- **策略模型**（Policy Model）：即 Actor Model，用于生成文本，并根据奖励信号不断优化；
- **参考模型**（Reference Model）：文本任务特有的模型，使用 SFT 模型初始化，防止其过度偏离初始点；
- **奖励模型**（Reward Model）：文本任务特有的模型，衡量整个序列的人类偏好，并分配到每个 Token 上；
- **价值模型**（Value Model）：即 Critic model，与配合 Reward 来估计优势和回报，从而引导策略更新。

1. 策略模型：

在RLHF中，策略模型一般使用与训练语言模型的SFT后的版本初始化，目标是通过生成符合人类偏好的文本来获得更高的奖励。

与PPO类似，RLHF中也会使用KL散度限制策略模型更新幅度，但不会直接使用上面PPO中显式约束作为优化目标，而是使用**奖励模型 计算完 KL散度 后直接与 Reward 结合**：

$$r_{\text{total}} = r(x, y) - \beta D_{\text{KL}}(\pi(y|x), \pi_0(y|x))$$

其中， $r(x, y)$ 是奖励模型给出的Reward， β 是KL散度惩罚系数。为了有效计算策略模型和参考模型之间的KL散度，可以从**对数概率(log_prob)**角度实现这一过程：

$$D_{\text{KL}}(\pi(y|x), \pi_0(y|x)) = \sum_t (\log \pi(y_t, x, y < t) - \log \pi_0(y_t, x, y < t))$$

具体步骤如下：

1. 使用策略模型 π 进行**采样**：对输入 x 生成的输出 y 。
2. 计算**策略模型 π 的对数概率** (log_probs)：计算策略模型在每个时间步 (Token) 的对数概率。
3. 计算**参考模型 π_0 的对数概率** (ref_log_probs)：对**相同的输入输出**，使用参考模型计算逐Token的对数概率。
4. 计算 KL 散度：通过**对数概率之差累加**得到策略模型与参考模型之间的 KL 散度。
5. 整合奖励：Token 级的 KL 散度与奖励模型在输出 y 上序列级 Reward 组合，得到 r_{total} 。



- **log prob 就是语言模型对一个文本的对数概率 (log-probability)**，计算公式如下：

给定一个 prompt x 和一个回答 r ，我们将其拼接为：

$$x = \text{prompt}, r = \text{response} = (y_1, y_2, \dots, y_T)$$

语言模型 π 是一个条件概率分布：

$$\pi(r | x) = \pi(y_1, y_2, \dots, y_T | x) = \prod_{t=1}^T \pi(y_t | x, y < t)$$

取对数后 (log-prob) 就是：

$$\log \pi(r|x) = \sum_{t=1}^T \log \pi(y_t | x, y < t)$$

log prob=语言模型逐token的对数概率之和，反应模型认为“这句话作为回答的可能性”有多高，用来衡量：在prompt给定下，模型更倾向生成哪个答案

2. 参考模型：

使用**预训练语言模型的 SFT 后的版本**初始化，并且**全程冻结参数**不更新。参考模型只在采样经验后计算 log_prob 时用到。

3. 奖励模型：

奖励模型通常是一个经过特殊微调的语言模型，其输入是生成的文本，输出是一个标量值，表示该文本的「质量」或「符合人类偏好的程度」。在 PPO 训练阶段，奖励模型的参数通常是冻结的，不参与更新。

4. 价值模型：

价值模型在 RLHF 中负责估计每个状态的价值函数，扮演经典 PPO 算法中的 Critic Model 角色。其作用是预测当前状态下（前缀序列）未来累积奖励的期望值，记为期望总收益 V_t 。这个 V_t 则在采样时为策略模型提供优势函数，帮助策略模型更好地理解哪些动作能够带来更高的奖励。可以是参考模型加一个线性层。

第1阶段：初始化阶段

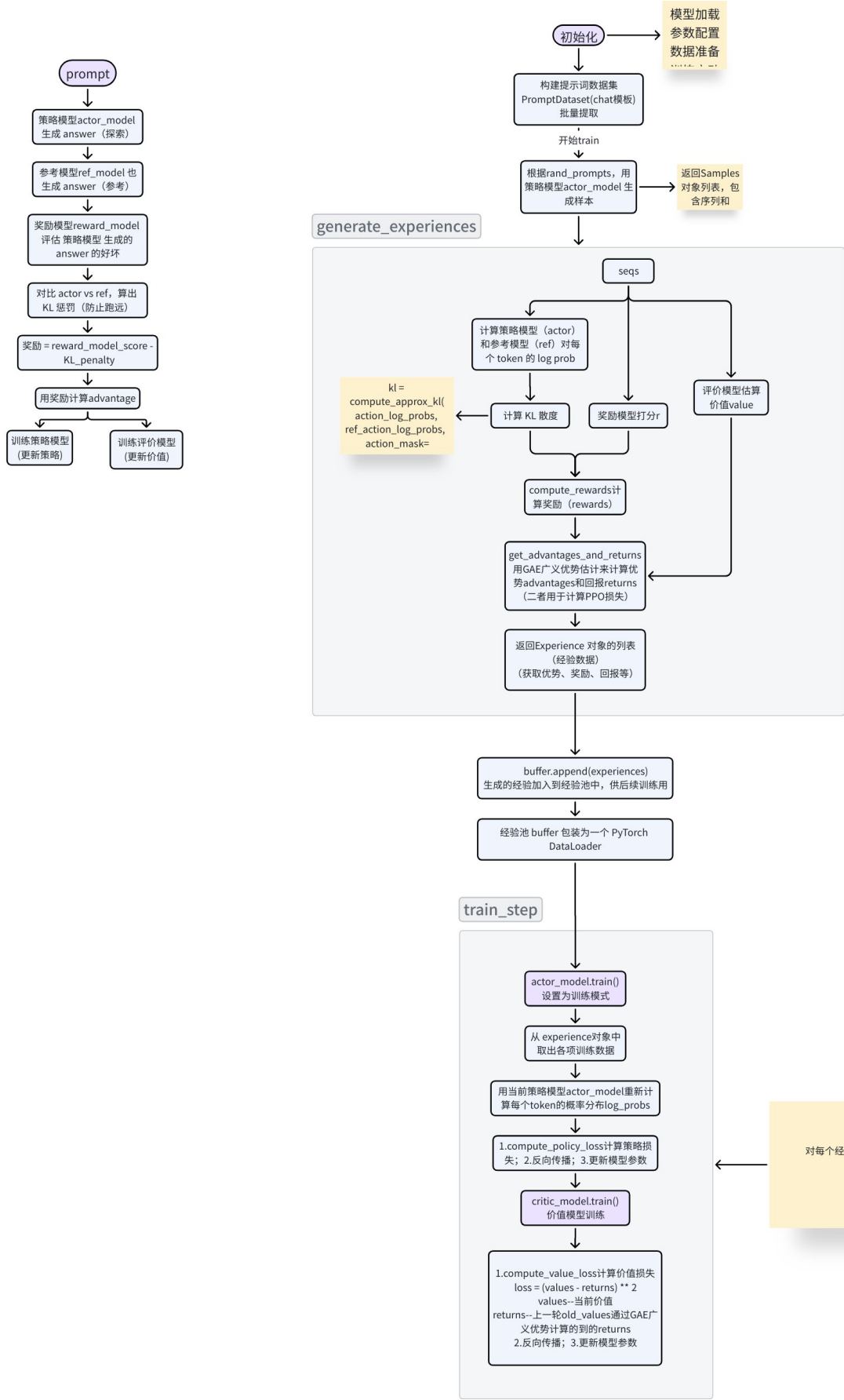
- 策略模型 π_θ ：通常初始化为经过监督微调(SFT)的大语言模型
- 参考模型 π_{ref} ：通常是SFT模型的副本
- 奖励模型 r ：人类偏好对比数据训练奖励模型，得到评分器RM
- 价值模型 V
- 超参数： β

第2阶段：PPO 训练策略模型（核心）

- step1.语言模型作为策略 π （初始就是 π_0 ，PPO中逐步更新优化。初始策略 π_0 是PPO参考基线，冻结不训练），生成回答
- step2.用 RM 评估回答质量，作为奖励
- step3.计算优势（Advantage）： $advantage = Return - \text{状态价值函数}$

$$A_\pi(s_t, a_t) = Return - V_\pi(s_t)$$

- step4.使用 PPO 算法优化语言模型 π （策略模型），不断更新策略模型，使高reward答案概率变大
- step5.重复上述步骤若干 epoch，直到模型输出更加符合奖励偏好（收敛）。

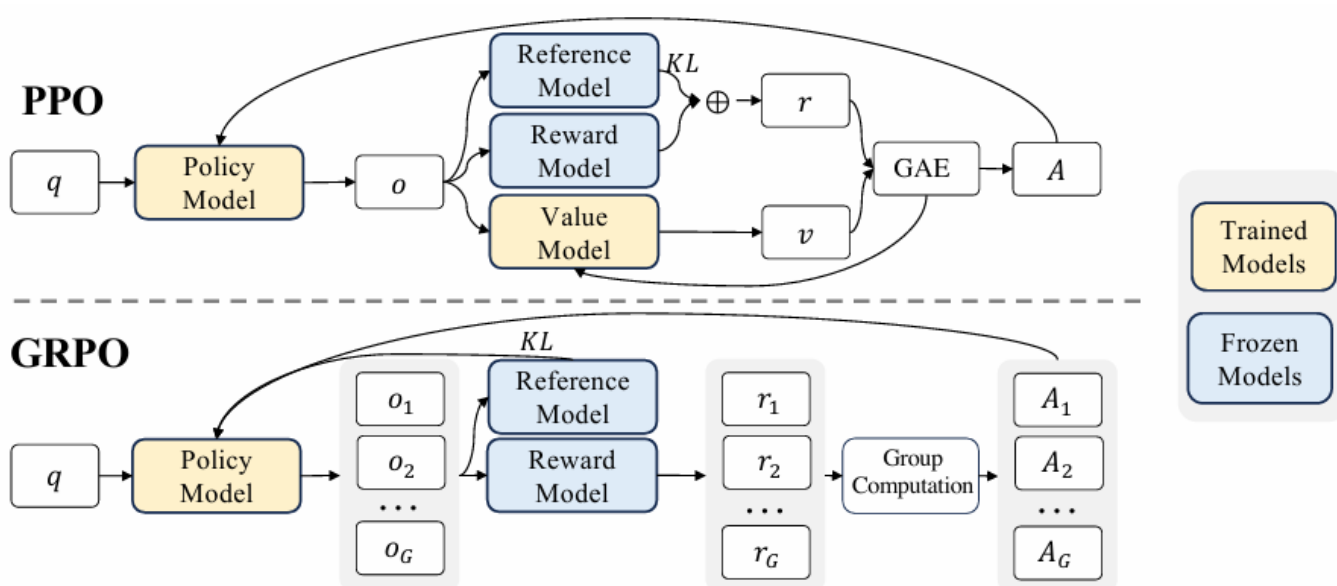


二、GRPO组相对策略优化

2024.4 DeepSeekMath 7B

GRPO的提出旨在解决PPO在大语言模型训练中的效率和稳定性问题，特别适合于计算资源受限或需要高效训练的场景。，提出于2024年。它通过引入**多样性对比机制**和 **KL 正则项**，在保持偏好对齐的同时**增强模型生成的多样性与稳定性**。

1. 完全避开价值网络：GRPO不再需要训练价值网络，大幅减少计算和内存开销。
2. 群体相对评估：针对同一问题，生成多个不同的回答，通过比较这些回答之间的相对质量来计算优势，而不是依赖绝对值预测。
3. 与**奖励模型**本质匹配：RLHF中的奖励模型本身就是通过比较同一问题下不同回答的相对优劣进行训练的，GRPO的相对评估机制与此天然匹配。
4. **KL散度正则化**改进：改进了KL散度控制方式，使训练更加稳定。



第1阶段：初始化阶段

- 策略模型 π_θ ：通常初始化为经过监督微调(SFT)的大语言模型
- 奖励模型 r_ϕ ：一个能评估回答质量的模型（自己设计的奖励函数）
- 参考模型 π_{ref} ：通常是SFT模型的副本，用于KL散度计算
- 超参数：裁剪参数 ϵ 、KL系数 β 、组大小 G 等

注意：不需要价值网络，这是GRPO相对于PPO的重要简化。

第2阶段：收集好**组排序**偏好数据

针对每个问题 q ，执行以下步骤：

1. 使用当前策略 π_θ 生成 G 个不同的回答
2. 记录每个回答中每个token的生成概率 $\pi_\theta(o_{i,t}|q, o_{i<t})$
3. 对于结果监督，使用奖励模型对每个完整回答评分，得到 $\tilde{r}_1, \tilde{r}_2, \dots, \tilde{r}_G$

4. 对于过程监督，对每个回答的推理步骤评分，得到更详细的奖励信号

代码块

```
1  {
2    "prompt": "解释机器学习与深度学习的区别",
3    "responses": [
4      "机器学习是人工智能的一个子领域...",
5      "深度学习是一种特定的机器学习方法...",
6      "它们都属于 AI，但深度学习更强大..."
7    ],
8    "ranking": [0, 2, 1] # 人类认为 responses[0] 最好，responses[1] 最差
9  }
10 # 可选扩展为 soft-label (如评分分数)，但常用的是排序 index。
```

第3阶段：计算相对优势--根据选择的监督类进行计算优势，GRPO支持两种监督方式，分别使用不同的优势计算方法-----结果监督&过程监督

1. 结果监督：只对最终输出进行评估

- 对每个问题 q ，从旧策略模型 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ ，然后使用**奖励模型**对输出进行评分，相应地产生 G 个奖励 $r = \{r_1, r_2, \dots, r_G\}$
- 每个奖励减去组平均值除以组标准差，对组内奖励进行标准化。结果监督在每个输出 o_i 的末尾提供标准化奖励，并将输出中所有token的优势 $\hat{A}_{i,t}$ 设置为标准化奖励

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}, \text{ 然后通过目标函数优化策略。}$$

2. 过程监督：对推理过程中的每个步骤进行评估

- 对每个问题 q ，从旧策略模型 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$ ，使用**过程奖励模型**对输出的每个步骤进行评分，产生相应的奖励：

$$R = \{\{r_1^{index(1)}, \dots, r_1^{index(K_1)}\}, \dots, \{r_G^{index(1)}, \dots, r_G^{index(K_G)}\}\}, \text{ 其中 } index(j) \text{ 是第 } j \text{ 个步骤的结束标记索引, } K_i \text{ 是第 } i \text{ 个输出中的总步骤数。}$$

- 还用均值和标准差标准化这些奖励， $\tilde{r}_i^{index(j)} = \frac{r_i^{index(j)} - \text{mean}(R)}{\text{std}(R)}$
- 过程监督将每个token的优势计算为后续步骤的标准化奖励之和： $\hat{A}_{i,t} = \sum_{index(j) \geq t} \tilde{r}_i^{index(j)}$ ，然后通过目标函数优化策略。

第4阶段：GRPO 训练策略模型（核心）

1. 为每个小批量计算GPRO目标函数：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E} \left[q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q) \right] \\ \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{\theta_{old}}] \right\}$$

与PPO的关键区别在于：

- 群体采样：针对每个问题 q ，从旧策略中采样 G 个不同的输出（通常 $G=64$ ）
- 优势计算方式： $\hat{A}_{i,t}$ 不再依赖于价值网络，而是基于组内的相对奖励计算
- KL散度正则化：直接在目标函数中添加KL散度项，而不是在奖励中添加KL惩罚
- 无偏KL估计：使用更准确的KL散度估计：

$$\mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1$$

2. 计算梯度并使用优化器（通常是Adam）更新策略模型参数

3. 对于每个问题的群组数据，进行多次更新（通常1-5次）

第5阶段：奖励模型迭代更新（可选）

如果使用迭代GRPO：

- 使用当前策略模型生成新数据
- 结合少量历史数据（10%）更新奖励模型
- 将当前策略模型设为新的参考模型
- 使用更新后的奖励模型继续训练策略模型

- 核心思路：对于每对(r_i, r_j)，若 r_i 排名高于 r_j ，则鼓励 $\log \pi(r_i | x) - \log \pi(r_j | x) > 0$

- 损失函数包含两部分：

- 1.相对偏好对比损失(Group Ranking Loss): 类似DPO的pair-wise softmax，对所有排序对(i,j)

构造损失项：
$$\mathcal{L}_{ranking} = - \sum_{(i,j) \in \mathcal{P}} \log \sigma(\beta \cdot (\log \pi(r_i | x) - \log \pi(r_j | x)))$$

$\log \pi(r_i | x)$ 是策略模型 π 对回答 r_i 的 $\log prob$ ；

β 是温度超参数，控制优化幅度（常设为0.1至1）；

σ 是sigmoid函数，用于将对比较差转化为概率；

$\mathcal{P}$ 是所有 $r_i \succ r_j$ 的配对集合(按 $ranking$)；

- 2.策略偏移正则项(KL Penalty)：限制策略 π 偏离 π_0 ： $\mathcal{L}_{KL} = \lambda \cdot D_{KL}(\pi || \pi_0)$

- 控制新策略 π 不能偏离 π_0 太远，防止学坏或输出奇怪答案，所以加一个KL正则项，这个KL项 惩罚新策略和旧策略差别太大。

- λ 是KL惩罚系数，控制惩罚强度（常设为0.1至1）

- **最终GRPO损失：** $\mathcal{L}_{GRPO} = \mathcal{L}_{ranking} + \mathcal{L}_{KL}$

GSPO分组序列策略优化

与以往采用**token 级重要性比率**的算法不同，GSPO 基于**序列似然性**定义重要性比率，并执行序列级的裁剪、奖励分配与优化

GRPO存在稳定性问题，其不稳定性源于其算法设计中对**重要性采样权重的根本性误用与失效**：这会引入高方差的训练噪声，且该噪声会随响应长度的增加而逐渐累积，并被裁剪机制进一步放大，最终导致模型崩溃。

GSPO 的核心创新在于：基于序列似然性，从理论层面严谨定义重要性比率，使其符合重要性采样的基本原理；同时，GSPO 将归一化奖励计算为“同一查询下多个响应的优势值”，确保序列级奖励分配与优化过程的一致性。

优化目标的单位应该和奖励的单位保持一致，

$$\mathcal{J}_{GSPO}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip} (s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right],$$

where we adopt the group-based advantage estimation:

$$\hat{A}_i = \frac{r(x, y_i) - \text{mean} (\{r(x, y_i)\}_{i=1}^G)}{\text{std} (\{r(x, y_i)\}_{i=1}^G)},$$

and define the importance ratio $s_i(\theta)$ based on sequence likelihood (Zheng et al., 2023):

$$s_i(\theta) = \left(\frac{\pi_{\theta}(y_i|x)}{\pi_{\theta_{old}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{old}}(y_{i,t}|x, y_{i,<t})} \right).$$

```
def gspo_loss(
    ref_policy_log_probs: torch.Tensor,
    old_policy_log_probs: torch.Tensor,
    new_policy_log_probs: torch.Tensor,
    attention_mask: torch.Tensor,
    advantages: torch.Tensor,
    prefix_len: int,
    clip_epsilon: float=0.2,
    kl_beta: float=0.04
):
    batch_size = ref_policy_log_probs.shape[0]

    # 取生成部分的概率分布
    ref_policy_log_probs_ = ref_policy_log_probs[:, prefix_len-1:] # token_0被裁剪了, 因此需要裁剪的长度为prefix_len-1
    old_policy_log_probs_ = old_policy_log_probs[:, prefix_len-1:]
    new_policy_log_probs_ = new_policy_log_probs[:, prefix_len-1:]
    attention_mask_ = attention_mask[:, prefix_len:] # attention_mask维度中token_0的位置没裁剪, 因此需要裁剪的长度为prefix_len

    # 计算有效序列, 遮挡pad_token
    valid_seq_len = attention_mask_.sum(dim=1)
    new_old_log_probs_ = (new_policy_log_probs_ - old_policy_log_probs_) * attention_mask_
    ref_new_log_probs_ = (ref_policy_log_probs_ - new_policy_log_probs_) * attention_mask_

    # 序列级别的重要性采样
    importance_ratio = torch.exp(new_old_log_probs_.sum(dim=1) / valid_seq_len).view(batch_size, 1) # batch_size * 1
    clipped_ratio = torch.clip(importance_ratio, 1 - clip_epsilon, 1 + clip_epsilon) # batch_size * 1
    importance_term = importance_ratio * advantages # batch_size * 1
    clip_term = clipped_ratio * advantages # batch_size * 1

    kl_term = torch.exp(ref_new_log_probs_.sum(dim=1) / valid_seq_len) - (ref_new_log_probs_.sum(dim=1) / valid_seq_len) - 1
    kl_term = kl_term.view(batch_size, 1)

    objective_function = torch.min(importance_term, clip_term) - kl_beta * kl_term
    sequence_loss = -objective_function

    # 批次平均损失作为总损失
    loss = sequence_loss.mean()
    return loss
```

第4阶段: GRPO 训练策略模型 (核心)

1. 为每个小批量计算GRPO目标函数:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E} \left[q \sim P_{sft}(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q) \right]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}$$

与PPO的关键区别在于:

- 群体采样: 针对每个问题 q , 从旧策略中采样 G 个不同的输出 (通常 $G=64$)
- 优势计算方式: $\hat{A}_{i,t}$ 不再依赖于价值网络, 而是基于组内的相对奖励计算
- KL散度正则化: 直接在目标函数中添加KL散度项, 而不是在奖励中添加KL惩罚
- 无偏KL估计: 使用更准确的KL散度估计:

$$\mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1$$

```
def grpo_loss(
    ref_policy_log_probs: torch.Tensor,
    old_policy_log_probs: torch.Tensor,
    new_policy_log_probs: torch.Tensor,
    attention_mask: torch.Tensor,
    advantages: torch.Tensor,
    prefix_len: int,
    clip_epsilon: float=0.2,
    kl_beta: float=0.04
):
    # 取生成部分的概率分布
    ref_policy_log_probs_ = ref_policy_log_probs[:, prefix_len-1:] # token_0裁剪了, 因此需要裁剪的长度为prefix_len-1
    old_policy_log_probs_ = old_policy_log_probs[:, prefix_len-1:]
    new_policy_log_probs_ = new_policy_log_probs[:, prefix_len-1:]
    attention_mask_ = attention_mask[:, prefix_len:] # attention_mask维度中token_0的位置没裁剪, 因此需要裁剪的长度为prefix_len

    importance_ratio = torch.exp(new_policy_log_probs_ - old_policy_log_probs_)
    clipped_ratio = torch.clip(importance_ratio, 1 - clip_epsilon, 1 + clip_epsilon)
    importance_term = importance_ratio * advantages
    clip_term = clipped_ratio * advantages

    kl_term = torch.exp(ref_policy_log_probs_ - new_policy_log_probs_) - (ref_policy_log_probs_ - new_policy_log_probs_) - 1

    objective_function = torch.min(importance_term, clip_term) - kl_beta * kl_term
    per_token_loss = -objective_function # 逐token损失: 负号表示最小化目标函数等价于最大化损失

    loss = ((per_token_loss * attention_mask_).sum(dim=1) / attention_mask_.sum(dim=1)).mean()
    return loss
```

三、DAPO直接对齐偏好优化（GRPO2.0版本）

2025.3 DeepSeek R1

在训练长CoT推理模型时，模型分布可能与初始模型有明显差异，因此没有必要进行这种限制。因此，将从提出的算法中排除KL项。解耦裁剪与动态采样策略优化算法DAPO对每个问题 q 及其对应答案 a 采样一组输出 $\{o_i\}_{i=1}^G$ ，对于每个问题 q 配对答案一个，并通过目标优化策略。

在GRPO基础上进行的改进：

1. 去掉KL散度

KL 惩罚项用于调节在线策略和冻结参考策略之间的散度。在人类反馈强化学习（RLHF）场景下，强化学习的目标是使模型行为与初始模型保持一致，而不过度偏离。然而，在训练长思维链推理模型时，模型分布可能会与初始模型显著偏离，因此这一限制并非必需，在提出的算法中去除了 KL 惩罚项。

2. 裁剪偏移-提高

clip 机制抑制了低概率Token的增长，DAPO将下裁剪范围 ϵ_{low} 保持为0.2（维持不变，和GRPO一样），上裁剪范围 ϵ_{high} 设置为0.28，从而在保持稳定性的同时提升策略的多样性。

3. 动态采样

对于GPRO中，某个prompt的所有输出都是正确的，奖励都等于 1 时，那么该组的结果优势为零，零优势导致策略更新没有梯度，导致策略更新的梯度消失，进而降低样本效率。实验也表明，准确率为1的样本数量会不断增加，每个批次中有效prompt的数量不断减少，这可能导致梯度方差更大，并削弱模型训练的梯度信号。

所以，DAPO 通过动态采样，如果一个问题的所有回复都正确or错误，那么就换一个问题，或者直到采样错误or正确的回复为止。。持续过滤全对/全错样本，实际上构建了一个「中等难度」训练集，这样可以保证每个 batch 中的样本都能对模型的训练产生积极作用。

$$s.t \quad 0 < |\{o_i \mid is_equivalent(a, o_i)\}| < G$$

这个公式代表正确回答的数量必须大于 0 同时小于所有回复的数量G。

4.基于token级的策略梯度损失

GPRO是样本级别计算损失，。这种方法导致所有样本在最终损失计算中具有相同的权重，在长思维链情况下，会导致两个问题：1.高质量的长样本对训练的贡献降低。2.低质量的长样本无法有效被惩罚。所以引入token级别的策略梯度损失,对所有token的损失求平均，确保每个token对策略优化的贡献更均衡

$$\mathcal{J}_{DAPO}(\theta) = \mathbb{E} \left[(q, a) \sim D, \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|q) \right] \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon_{low}, 1 + \epsilon_{high} \right) \hat{A}_{i,t} \right) \right]$$

5.对过长回答修改奖励

强化学习训练的时候会设定个最大生成长度，超长样本会被截断，在loss中屏蔽对超长文本的惩罚，常规的做法是对超长的文本直接给一个负分Reward，但是这样可能会存在文本推理是高的，只不过由于各种原因（例如难题）超长了，但是这时候给一个负向Reward可能会对优化方向有一定影响。因此本文还对超长文本在loss级别进行了mask，防止他们影响优化方向。

同时给出软惩罚。当输出长度接近最大长度时，逐步给予长度惩罚，防止输出过长。

超长惩罚公式：（论文里面 L_{cache} 设置为4096， L_{max} 为20480）

$$R_{length}(y) = \begin{cases} 0, & |y| \leq L_{max} - L_{chche} \\ \frac{(L_{max} - L_{chche}) - |y|}{L_{chche}}, & L_{max} - L_{chche} < |y| \leq L_{max} \quad \text{过渡区} \\ -1, & |y| > L_{max} \end{cases}$$

- L_{max} 是模型输出长度上限，超过这个长度就直接给到最大惩罚；
- $L_{max} - L_{chche}$ 是理想长度，低于这个长度不予惩罚；
- L_{cache} 是模型输出长度的一个类似“缓存空间”的变量。意思就是说，当模型输出的长度从理想长度 $L_{max} - L_{chche}$ 开始逼近最大长度 L_{max} 时，给予从 0 到 -1 的线性惩罚

比如，，那么模型输出 90 个 token 以内视作合理，一旦超过 90 个就开始给惩罚，比如 91 个 token 就给 -0.1 的惩罚。

第1阶段：初始化阶段

- 策略模型 π_{θ} ：通常初始化为经过监督微调(SFT)的大语言模型
- 奖励模型 r_{φ} ：一个能评估回答质量的模型（自己设计的奖励函数）

- 超参数：裁剪参数 ϵ_{low} ϵ_{high} 、组大小 G 等)

第2阶段：准备好偏好对比数据，执行以下步骤

- 采样批次数据：从数据集 $\mathcal{D}$ 中采样一个批次的任务提示 $\mathcal{D}_b$
- 更新旧策略：将当前策略模型 π_θ 复制为旧策略模型 $\pi_{\theta_{old}}$ ，用于后续重要性采样计算。
- 生成候选输出：针对每个提示 $q \in \mathcal{D}_b$ ，通过旧策略 $\pi_{\theta_{old}}$ 采样 G 个输出 $\{o_i\}_{i=1}^G$ （即不同的推理路径）
- 计算奖励：通过奖励模型 R 为每个输出 o_i 计算奖励 $\{r_i\}_{i=1}^G$ 。奖励基于规则（如答案正确性），正确为 1，错误为 -1
- 动态采样过滤：过滤掉奖励为 0 或 1 的样本（即完全正确或完全错误的输出），仅保留能提供有效梯度的样本到动态采样缓冲区。若缓冲区未填满（小于 N ），则继续采样。

代码块

```
1  {
2    "prompt": "介绍深度学习与传统机器学习的区别",
3    "chosen": "深度学习是基于神经网络的自动特征提取方法...",
4    "rejected": "深度学习就是计算能力更强..."
5  }
```

第3阶段：计算优势估计

对缓冲区中每个输出 o_i 的每个 token，计算优势估计 $\hat{A}_{i,t}$ ，通过组内奖励归一化实现（参考 GRPO 的优势计算方式）
$$\hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}$$

第4阶段：DAPO 训练策略模型（核心）

在多个迭代（iteration = 1,..., μ ）中，通过最大化 DAPO 的目标函数更新策略模型 π_θ 。目标函数结合了解耦裁剪（Clip-Higher）和 token 级梯度损失

DAPO目标函数：

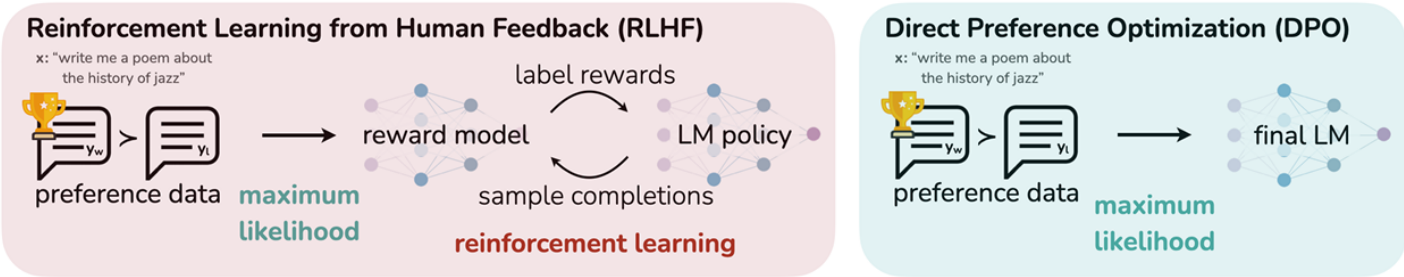
$$\mathcal{J}_{DAPO}(\theta) = \mathbb{E} \left[(q, a) \sim D, \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot | q) \right] \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | q, o_{i,<t})}, 1 - \epsilon_{low}, 1 + \epsilon_{high} \right) \hat{A}_{i,t} \right) \right]$$

$$s.t \quad 0 < |\{o_i \mid is_equivalent(a, o_i)\}| < G$$

其中，
$$\hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}$$

四、DPO直接偏好优化

直接优化语言模型以遵循人类偏好，而无需显式奖励建模或强化学习。给定人类偏好数据集，DPO 可以使用简单的二元交叉熵目标来优化策略，从而生成适合偏好数据的隐式奖励函数的最优策略。



第1阶段：初始化阶段

- 策略模型 π_θ ：通常初始化为经过监督微调(SFT)的大语言模型
- 参考模型 π_{ref} ：通常是SFT模型的副本
- 超参数： β

第2阶段：准备好偏好对比数据

代码块

```
1  {
2    "prompt": "请介绍一下量子纠缠（输入）",
3    "chosen": "量子纠缠是指两个粒子在状态上紧密关联...（偏好回答）",
4    "rejected": "量子力学是研究电子的学科...（不优回答）"
5  }
```

第3阶段：DPO 训练策略模型（核心）

通过极大似然估计，将 BT偏好模型的优化目标转化为二元交叉熵损失：

$$\mathcal{L}_{DPO}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

- x 是prompt, y_w 是chosen, y_l 是rejected
- $\pi_\theta(y_w|x)$ 是当给定输入x时，当前策略模型生成好的答案的概率
- $\pi_{ref}(y_w|x)$ 是当给定输入x时，参考模型生成好的答案的概率
- β 是温度超参数（控制分布平滑度，常设为0.1至1）
- σ 是sigmoid函数

这个损失函数具有三个关键特性：

- a. 隐式奖励建模：通过策略网络与参考策略的比值隐式学习奖励函数，避免显式奖励模型的偏差；

- b. **数据高效性**：直接利用成对偏好数据，无需从策略网络中采样（对比 PPO 需要 on-policy 采样）；
- c. **动态权重调节**： β 控制策略更新幅度，防止过度偏离参考策略（类似 PPO 中的 KL 约束）。
- 梯度下降优化策略模型,使用该loss对策略模型参数执行优化

代码块

```
1 loss.backward()
2 optimizer.step()
```

- 重复训练多个 epoch，直到模型明显倾向于偏好答案



DPO结构伪代码

代码块

```
1 # 模型初始化
2 model = AutoModelForCausalLM.from_pretrained("your_sft_model")
3 for batch in dataloader:
4     prompt = batch["prompt"]
5     chosen = batch["chosen"]
6     rejected = batch["rejected"]
7     # 拼接输入 (prompt+response)
8     input_chosen = tokenizer(prompt + chosen, return_tensors="pt",
padding=True)
9     input_rejected = tokenizer(prompt + rejected, return_tensors="pt",
padding=True)
10    # 获取 log_probs
11    log_prob_chosen = model.get_log_prob(input_chosen)
12    log_prob_rejected = model.get_log_prob(input_rejected)
13    # 计算 DPO 损失
14    loss = -torch.nn.functional.logsigmoid(beta * (log_prob_chosen -
log_prob_rejected)).mean()
15    # 更新策略模型
16    loss.backward()
17    optimizer.step()
```

五、RLEF（人类偏好反馈preference+专家动作监督expert）

RLEF 是一种融合偏好学习（如 DPO）与专家模仿（SFT 示例）的强化学习方法，不依赖奖励模型，能充分利用两种监督信号对策略模型进行稳定、可控、高质量优化。

第1阶段：SFT

- 用人类写的回答微调基础模型（得到初始语言模型 π_0 ，也叫初始策略模型）

第2阶段：准备两类数据

第1类：人类偏好数据（类似DPO），三元组 $(x, y+, y-)$

代码块

```
1  {
2    "prompt": "请介绍一下量子纠缠（输入）",
3    "chosen": "量子纠缠是指两个粒子在状态上紧密关联...（偏好回答）",
4    "rejected": "量子力学是研究电子的学科...（不优回答）"
5  }
```

第2类：专家示范数据 (x, y^*) ， x : prompt y^* : 高质量 expert 回复（来自 SFT 标注或 GPT-4 精选）

代码块

```
1  # 它和偏好对比数据  $(x, y+, y-)$  不同，不需要和差答案比较，仅提供“做得对”的示范。
2  {
3    "prompt": "请用通俗的语言解释什么是黑洞。",
4    "expert_answer": "黑洞是一种宇宙中非常神秘的天体，它的引力大到连光都无法逃脱。你可以想象成一个巨大的吸尘器，只要靠近它，任何东西都会被吸进去，甚至连光也不例外。"
5  }
```

第3阶段：RLEF 训练策略模型（核心）

- step1.sft得到的语言模型 π_0 ，它只是参考策略，不参与训练。
- step2.初始化策略模型 π （待优化模型），从 π_0 拷贝参数，作为DAPO优化起点。训练时只更新 π 的参数， π_0 保持冻结。

代码块

```
1  # 克隆 $\pi_0$ 的参数作为 $\pi$ 的初始值
2   $\pi = \text{copy.deepcopy}(\pi_0)$ 
```

- step3.策略模型计算概率 (log prob)
 - 使用当前语言模型 π (策略) 分别计算

代码块

```
1 log_prob_chosen =  $\pi$ (chosen | prompt)
2 log_prob_rejected =  $\pi$ (rejected | prompt)
3 # 可以看成是生成这两个回答的对数概率
```

- step4.计算RLEF损失函数 (核心)

优化语言模型，使它更偏向于生成 chosen，而不是 rejected

损失函数包括三部分：

1.偏好损失 (DPO)

$$\mathcal{L}_{\text{pref}} = -\log \sigma(\beta \cdot (\log \pi(y^+ | x) - \log \pi(y^- | x)))$$

- x 是prompt, y^+ 是chosen, y^- 是rejected
- π 是当前策略模型
- β 是温度超参数 (控制分布平滑度，常设为0.1至1)
- σ 是sigmoid函数

2.模仿专家损失

$$\mathcal{L}_{\text{imit}} = -\log \pi(y^* | x)$$

3.KL散度 (正则项)，限制 π 不要偏离 π_0

$$\mathcal{L}_{KL} = \lambda \cdot D_{KL}(\pi \| \pi_0) = \lambda \cdot D_{KL}(\pi(\cdot | x) \| \pi_0(\cdot | x))$$



- D_{KL} 是KL散度，用来衡量两个概率分布之间的“距离”或“差异”

$$\circ D_{KL}(\pi \| \pi_0) = \frac{1}{T} \sum_{t=1}^T \sum_{v \in V} \pi(v_t | x, y < t) \log \left(\frac{\pi(v_t | x, y < t)}{\pi_0(v_t | x, y < t)} \right)$$

- T : 回复 y 的总token数
- V : 词汇表vocabulary的所有token
- v_t : 第 t 个位置的候选token (遍历所有vocab)

RLEF总损失函数: $\mathcal{L}_{\text{RLEF}} = w_1 \cdot \mathcal{L}_{\text{pref}} + w_2 \cdot \mathcal{L}_{\text{imit}} + \lambda \cdot \mathcal{L}_{KL}$

常见超参数设置：

- $w_1 = 1.0, w_2 = 0.5$ 至 $1.0, \lambda = 0.01$ 至 0.1

- step5.训练模型 Π （反向传播，优化 Π ）

代码块

```
1  loss.backward()  
2  optimizer.step()  
3  optimizer.zero_grad()
```